Functions

What is Functions?

Reusable blocks of code designed to perform a specific, isolated task.

“Like a recipe abstracts cooking steps, a function abstracts code logic.”

WHY USE THEM?

1

Modularity

Execute tasks multiple times without duplicating instructions.

2

Readability

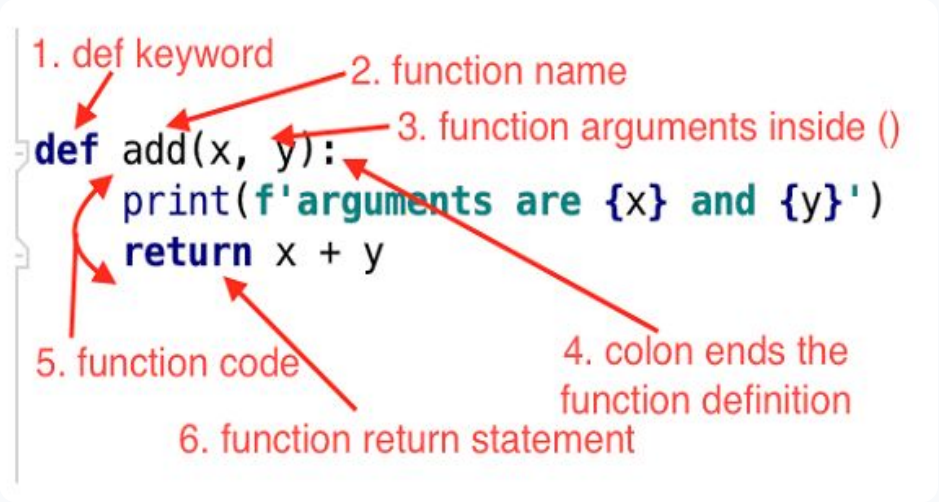
Break complex logic into smaller, understandable chunks.

3

Maintenance

Update logic in one place to affect all occurrences.

Anatomy of a Function



Understanding the Syntax

- **1. def keyword:** Marks the start of a function header.
- **2. Function Name:** A unique identifier to call the function.
- **3. Arguments(Parameters):** Inputs passed into the function inside ().
- **4. Colon (:):** Signal the end of the function header.
- **5. Function Code:** The block of logic that performs the task.
- **6. Return Statement:** Exits the function and passes back a value.

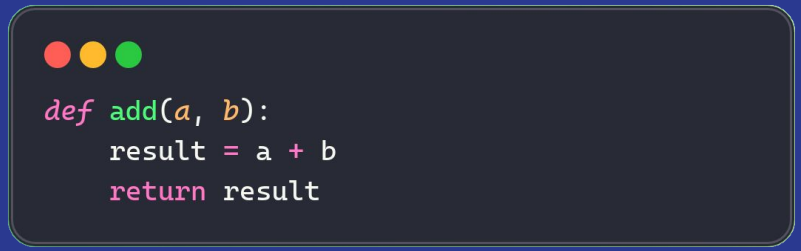
Define and Call a Function

Define a Function with def

1. Write **def** and name your function

2. Indent 4 or 2 spaces (must stay consistent), then write the body

3. Use **return** to send back a value (Optional)



USING FUNCTIONS

Calling a Function

DEFINE

```
def add(a, b):
    result = a + b
    return result
```

Use the function name + parentheses with arguments to run it.

```
def add(a, b):
    return a + b

result = add(5, 3)
print(result)  # → 8
```

Insight: the values you pass in (5, 3) are matched to (a, b) in order.

Named Argument

Argument can be passed by named argument

- Use name=value inside the parentheses when calling a function.
- Order no longer matters: (width=10, length=20) assigns width first, even though length is defined first.
- Prevents mix-ups: You don't have to memorize the exact parameter order.

Insight: the values you pass in (5, 3) are matched to (a, b) in order.

```python
def calculate_area(length, width):
    print(f"width: {width}")
    print(f"length: {length}")
    area = length * width
    return area # Optional for return
```

```python
total_area = calculate_area(10,20)
print(f"Total: {total_area}")
```

```
width: 20
length: 10
Total: 200
```

```python
total_area = calculate_area(width=10, length = 20)
print(f"Total: {total_area}")
```

```
width: 10
length: 20
Total: 200
```

Default Parameter Values

Give parameters a fallback value when no argument is passed

- parameter=value in **def** (e.g., width=5).
- **Optional to pass**: Uses the default automatically if left out.
- **Easy to override**: Pass a new value to replace the default.
- **Put at the end**: Default parameters must come after required ones.

Insight: Caution: Required parameters must come first! Writing def calculate_area(length=10, width): will cause a SyntaxError.

[2]:
```
def calculate_area(length, width=5):
    print(f"width: {width}")
    print(f"length: {length}")
    area = length * width
    return area
```

[3]:
```
# Only pass 'length' -> 'width' uses default (5)
total_area = calculate_area(10)
print(f"Total: {total_area}")
```

```
width: 5
length: 10
Total: 50
```

[4]:
```
# Pass both -> overrides default 'width' with 20
total_area = calculate_area(10, 20)
print(f"Total: {total_area}")
```

```
width: 20
length: 10
Total: 200
```

Function with and without **return**

Functions Can Return Values

return sends a result back to the caller.

```
def multiply(x, y):
    return x * y        # sends the answer back

answer = multiply(4, 5)
print(answer)           # output: 20
```

EXAMPLE OUTPUT

multiply(4, 5)
→ 20

Insight: a function without return gives back None by default.

Function with No Return

PRINT VS RETURN

Why it matters

- print() displays a value on screen — for humans, not the program
- return sends the value back so it can be stored, passed along, or computed with
- Here, multiply() only prints — the caller's result ends up None

```
def multiply(a, b):
    print(a * b)

result = multiply(4, 5)
print(result)
```

```
Out[1]:  20
         None
```

Highlighted: the print() inside the function, and result used outside

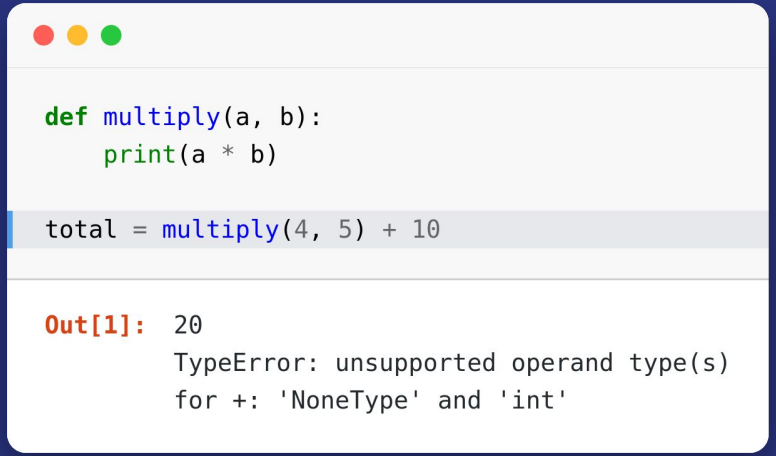
- **Caution:** 20 appears on screen, but result is still None — multiply() never returned anything.

Common mistakes

What goes wrong

- multiply() only prints — it never returns
- Without return, python will return None implicitly
- So total = multiply(4, 5) + 10 tries to add None + 10
- Python raises a TypeError because None isn't a number

Highlighted: the line that fails



- **Caution:** if you plan to use a function's result later, it must return it — not just print it.

Lambda Function

Lambda Function

Functions that don’t need to be named, defined without the standard def keyword.

```
[1]: s = lambda v1, v2: v1+v2

     s(10, 20)

[1]: 30
```

1

Anonymous Identity

No name is required — the function exists just long enough to be used, often passed directly as an argument.

2

Efficient One-Liners

Ideal for small operations where a full def block would be overkill for a simple, single-expression task.

Find event number using Function

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
def is_event(num):
    return num % 2 == 0

even_numbers = list(filter(is_event, numbers))
even_numbers
```

```
[2, 4, 6, 8, 10]
```

Find event number using Lambda

```
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
even_numbers
```

```
[2, 4, 6, 8, 10]
```

Sort by field using Function

[8]:
```
data = [
    {"name": "Alice", "age": 30},
    {"name": "Bob", "age": 25},
    {"name": "Charlie", "age": 35}
]
```

[11]:
```
def compared_field(person):
    return person["age"]

sorted_by_age = sorted(data, key=compared_field)
sorted_by_age
```

[11]:
```
[{'name': 'Bob', 'age': 25},
 {'name': 'Alice', 'age': 30},
 {'name': 'Charlie', 'age': 35}]
```

Sort by field using Lambda

```
sorted_by_age = sorted(data, key=lambda person: person["age"])
sorted_by_age
```

```
[{'name': 'Bob', 'age': 25},
 {'name': 'Alice', 'age': 30},
 {'name': 'Charlie', 'age': 35}]
```

Exercise

Level 1: Beginner

DEFINITIONS · INVOCATION · PRINT VS RETURN

Understanding function headers, basic parameters, and how return values work vs printing.

Exercise 1: Simple Greeting

```
def greet_user(name):
    # your code here
    pass

# call it twice with different names
```

Your task

- Goal: practice function syntax and parameter passing.
- Write `greet_user(name)` that prints:
  - "Hello, <name>! Welcome to Python."
- Call the function twice with different names.

Expected output:

[1] greet_user("Amy") → output: "Hello, Amy! Welcome to Python."

[2] greet_user("Liam") → output: "Hello, Liam! Welcome to Python."

Exercise 2: Area Calculator

```
def calculate_rectangle_area(width, height):
    # your code here
    pass

area = calculate_rectangle_area(4, 5)
print(area * 2)
```

Your task

- Write `calculate_rectangle_area(width, height)` that returns the area.
- Assign the result to `area`, then print `area * 2`.

Expected output:

[1] width=4, height=5 → output: 40
[2] width=3, height=6 → output: 36

Exercise 3: Fix the NoneType Bug

```
def add_tax(price):
    print(price * 1.1)

total = add_tax(100) + 5  # Currently fails!
```

Your task

- Goal: recognize and fix a `TypeError` caused by a missing `return`.
- `add_tax()` currently prints instead of returning a value.
- Fix the function so `total` calculates correctly.

Expected output:

[1] add_tax(100) + 5 → Output: 115.0
[2] add_tax(50) + 5 → Output: 60.0

Exercise 4: Even/Odd Checker

```
def is_even(number):
    # your code here
    pass

print(is_even(7))
print(is_even(12))
```

Your task

- Goal: use control flow inside a function.
- Write `is_even(number)` that returns True if even, False if odd.
- Test it with `7` and `12`.

Expected output:

[1] is_even(7) → Output: False
[2] is_even(12) → Output: True

Level 2: Intermediate

MULTI-PARAMETER FUNCTIONS · LAMBDAS

Handling multiple inputs, using default values, and writing basic inline anonymous functions.

Exercise 5: Format Full Names

```
def format_full_name(first_name, last_name, middle_name=""):
    # your code here
    pass


name = format_full_name("Ana","Lee")
print(name)
name = format_full_name("Ana","Lee","J")
print(name)
```

Your task

- Goal: handle multiple positional arguments and a default value cleanly.
- Return a clean full-name string from `first_name`, `last_name`, and an optional `middle_name`.
- Test it with and without a middle name.

Expected output:

[1] format_full_name("Ana","Lee") → Output: "Ana Lee"
[2] format_full_name("Ana","Lee","J") → Output: "Ana J Lee"

Exercise 6: Def to Lambda

```
def square(x):
    return x ** 2

print(square(5))
print(square(9))
```

Your task

- Goal: convert a standard `def` function into a concise `lambda`.
- Rewrite `square(x)` above as a single-line lambda assigned to a variable named `square`.

Expected output:

[1] square(5) → Output: 25
[2] square(9) → Output: 81

Exercise 7: Filter Even Numbers

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# your code here
```

Your task

- Goal: use lambdas for quick, inline evaluation.
- Given the list above, use a lambda inside `filter()` (or a list comprehension) to keep only the even numbers.

Expected output:

[1] numbers = [1..10] → return: [2, 4, 6, 8, 10]

Exercise 8: Discount Calculator

```
def apply_discount(price, discount_percent=10):
    # your code here
    pass

print(apply_discount(100))
print(apply_discount(100, 20))
```

Your task

- Goal: master optional parameters and default return behavior.
- Return the discounted price from `price` and `discount_percent` (default 10).
- Call it with just `price`, then override the discount with `20`.

Expected output:

[1] apply_discount(100) → Output: 90.0
[2] apply_discount(100, 20) → Output: 80.0

Level 3: Advanced

SORTING · COMPOSITION · LOGIC ABSTRACTION

Sorting complex data structures using key functions, chaining functions, and logic abstraction.

Exercise 9: Sort Students by Score

```
students = [
    {"name": "Alice", "score": 88},
    {"name": "Bob", "score": 95},
    {"name": "Charlie", "score": 78}
]

# your code here
```

Your task

- Goal: use lambdas as key functions for sorting complex data.
- Sort `students` in descending order by `"score"` using `sorted()` and a lambda key.

Expected output:

[1] sorted by score, descending → return: [Bob 95, Alice 88, Charlie 78]

Exercise 10: Function Pipeline

```
def clean_text(text):
    # your code here
    pass

def count_words(text):
    # your code here
    pass

def process_document(raw_text):
    # your code here
    pass
```

Your task

- Goal: combine functions where one's return value feeds the next.
- `clean_text(text)` → strips whitespace, lowercases.
- `count_words(text)` → returns word count.
- `process_document(raw_text)` → calls `clean_text`, then `count_words`.

Expected output:

[1] process_document(" Hello WORLD ") → return: 2

Extra Learning resource

- https://code.tutsplus.com/introduction-to-python--CRS-200307c
- https://code.tutsplus.com/data-handling-with-python--CRS-200572c
- https://gto76.github.io/python-cheatsheet/

